



Univerzitet u Novom Sadu
Tehnički fakultet "Mihajlo Pupin"
Zrenjanin



Flagovi u procesoru i njihova uloga

Predmet: Operativni sistemi

Profesor: Prof. dr Ljubica Kazi

Student: Miroslav Zvara

Broj indeksa: SI 3/24

Zrenjanin, 2026.

Sadržaj

1. Teorijski deo	3
1.1 Teorijske osnove	3
1.1.1 Arhitektura procesora	3
1.1.2 Registri procesora	3
1.1.3 Definicija i uloga flagova	4
1.1.4 Vrste flagova	4
1.1.5 Karakteristike flagova	5
1.2 Tehnološke osnove	6
1.2.1 ALU i generisanje flagova	6
1.2.2 Statusni registri i organizacija flagova	6
1.2.3 Flagovi u assembleru	7
1.2.4 Uticaj flagova na izvršavanje programa	7
1.3 Komparativna analiza postojećih rešenja	7
1.3.1 Flagovi u ARM arhitekturi	7
1.3.2 RISC arhitekture i alternativni pristupi	8
1.3.3 Poređenje arhitektura	9
1.4 Primeri primene	9
1.4.1 Kontrola toka programa	9
1.4.2 Aritmetičke operacije	12
1.4.3 Operativni sistemi	13
1.4.4 Embedded sistemi i mikrokontroleri	14
1.4.5 Debugovanje i reverzni inženjering	14
2. Praktični deo	15
2.1 Namena softvera i korisničko uputstvo	15
2.2 Kratak opis primenjenih alata i tehnologija u implementaciji rešenja	17
2.3 Opis implementacije	18
3. Zaključak	30
4. Literatura	31

1. Teorijski deo

1.1 Teorijske osnove

1.1.1 Arhitektura procesora

Procesor (CPU - Centralna procesorska jedinica) je glavni deo računara. Njegova funkcija je izvršavanje programa sačuvanih u glavnoj memoriji sekvencijalnim izvršavanjem instrukcija. Komponente CPU su povezane magistralom, koja je skup paralelnih žica koje se koriste za prenošenje adresa, podataka i kontrolnih signala. [1]

CPU se sastoji od nekoliko različitih delova. Kontrolna jedinica je zadužena za preuzimanje instrukcija iz glavne memorije i određivanje njihove vrste. Aritmetičko-logička jedinica (ALU) izvršava operacije kao što su sabiranje i bulovo I, koje su potrebne za izvršavanje instrukcija. CPU takođe sadrži malu memoriju visokih brzina koja čuva privremene rezultate i kontrolne podatke. Ova brza memorija se sastoji od registara, svaki od njih ima određenu funkciju. [1]

Izvršavanje instrukcija zahteva proces koji se naziva „instrukcioni ciklus“. Instrukcioni ciklus se sastoji od tri osnovne faze: preuzimanja instrukcije (fetch), dekodiranja instrukcije (decode) i izvršavanja instrukcije (execute). U fazi preuzimanja se instrukcija učitava iz glavne memorije u procesor. Tokom faze dekodiranja se određuje vrsta instrukcije i operandi koji će biti korišćeni. Nakon toga, u fazi izvršavanja procesor izvršava odgovarajuću operaciju i upisuje rezultat. [2]

Postoje mnoge arhitekture procesora koje se danas koriste. CISC i RISC predstavljaju dve osnovne filozofije dizajna instrukcijskih skupova procesora. CISC arhitekture karakteriše veći broj kompleksnijih instrukcija koje mogu izvršiti više operacija u jednoj instrukciji, dok RISC arhitekture koriste manji skup instrukcija koji je optimizovan za brže izvršavanje i efikasnije procesiranje. [3]

Arhitektura x86 predstavlja kompleksan CISC instrukcijski skup koji definiše način na koji procesor izvršava instrukcije, upravlja registrima i koristi sistem memorije. Ova arhitektura razvijena je od strane Intel-a i široko je korišćena u ličnim računarima i serverskim sistemima. Karakteriše je veliki broj instrukcija različite složenosti. [4]

ARM arhitektura predstavlja skup RISC instrukcija i pravila koji definišu način izvršavanja programa, organizaciju registara i model obrade podataka u procesoru. Karakteriše je jednostavniji i efikasniji skup instrukcija u odnosu na CISC arhitekture. Ovo omogućava nižu potrošnju energije i efikasno izvršavanje instrukcija u embedded i mobilnim sistemima. [5]

1.1.2 Registri procesora

Registri predstavljaju najmanje, ali najbrže memorijske jedinice unutar centralne procesorske jedinice. Koriste se za privremeno čuvanje podataka, memorijskih adresa i kontrolnih signala neophodnih za izvršavanje instrukcija, kao i za skladištenje rezultata prethodno izvršenih operacija. Njihova blizina procesorskoj jedinici omogućava izuzetno brz pristup podacima. [2]

Registri se mogu podeliti na registre opšte namene, registre specijalne namene i adresne registre. Registri opšte namene se koriste u programima za skladištenje operanada i rezultata i mogu se adresirati kao 8-bitni, 16-bitni, 32-bitni ili 64-bitni. U x86 arhitekturi, 16-bitni registri opšte namene su AX, BX, CX, DX, SI, DI i BP. 32-bitni registri imaju prefiks E(extended), dok 64-bitni registri imaju prefiks R. [6]

Registri specijalne namene obuhvataju pokazivač na instrukciju (IP), pokazivač na stek (SP), statusni registar (FLAGS) i segmentne registre (CS, DS, ES, SS, FS i GS).

IP registar pokazuje na memorijsku lokaciju instrukcije koja se izvršava sledeća. SP registar se koristi pri indeksiranju steka procesora i pokazuje na vrh steka. FLAGS registar predstavlja indikator stanja procesora nakon svake instrukcije. Segmentni registri sadrže adresu početka odgovarajućeg segmenta u memoriji i koriste se za generisanje adresa u memoriji. [6]

1.1.3 Definicija i uloga flagova

Flagovi predstavljaju posebne bitove u statusnom registru procesora koji opisuju rezultat prethodno izvršene aritmetičke ili logičke operacije. Oni ne čuvaju podatke sami po sebi, već informacije o stanju procesora nakon izvršavanja instrukcije. Nalaze se u statusnom registru procesora (x86 arhitektura ih naziva FLAGS, EFLAGS ili RFLAGS), koji predstavlja deo procesorskog registra za kontrolu i praćenje stanja izvršavanja instrukcija. [4]

Flagovi nastaju automatski kao rezultat rada aritmetičko-logičke jedinice. Nakon izvršenja instrukcije kao što je sabiranje, oduzimanje ili logičke operacije, procesor ažurira određene bitove u statusnom registru u zavisnosti od rezultata operacije. [4]

Uloga flagova je ključna u donošenju odluka tokom izvršavanja programa, jer omogućavaju uslovno grananje i kontrolu toka programa u assembleru. [4]

1.1.4 Vrste flagova

Najčešće korišćeni flagovi u x86 arhitekturi uključuju Carry, Zero, Sign, Overflow, Parity i Auxiliary Carry. Ovi flagovi se automatski postavljaju ili brišu u zavisnosti od rezultata operacije u aritmetičko-logičkoj jedinici procesora. Pored statusnih flagova, postoje i kontrolni flagovi koji utiču na način izvršavanja instrukcija, kao što su Interrupt Flag, Direction Flag i Trap Flag. Oni omogućavaju kontrolu nad prekidima, radom sa stringovima i režimom debugovanja. [6]

Flag	Oznaka	Opis
Carry	CF	Prenos ili pozajmica u aritmetičkim operacijama
Auxiliary Carry	AF	Pomoćni prenos ili pozajmica
Zero	ZF	Rezultat operacije jednak nuli
Sign	SF	Predznak rezultata (dvojni komplement)

Overflow	OF	Prekoračenje opsega za označene (signed) brojeve
Parity	PF	Paritet rezultata (paran/neparan broj jedinica u rezultatu)
Interrupt	IF	Dozvola hardverskih prekida
Direction	DF	Pravac string operacija (inkrement/dekrement)
Trap	TF	Omogućava debug režim

Tabela 1 – Opis najčešće korišćenih flagova u x86 arhitekturi [6]

1.1.5 Karakteristike flagova

Procesorski flagovi imaju specifične karakteristike koje ih razlikuju od drugih registara u procesoru i definišu njihovu ulogu u izvršavanju instrukcija. [4]

Flagovi se ne menjaju direktno od strane programera, već ih automatski ažurira aritmetičko-logička jedinica nakon izvršavanja instrukcija. Svaka aritmetička ili logička operacija može promeniti određene bitove u statusnom registru u zavisnosti od rezultata. [4]

Vrednosti flagova su privremene i odnose se samo na rezultat poslednje izvršene instrukcije koja ih je promenila. Svaka naredna instrukcija koja utiče na statusni registar može prepisati prethodno stanje flagova. [4]

Instrukcije prenosa podataka često nemaju efekat na statusne bitove. Zbog ovoga flagovi direktno zavise od tipa izvršene instrukcije. [4]

Flagovi omogućavaju razlikovanje operacija nad označenim i neoznačenim brojevima. Carry flag se koristi u neoznačenoj aritmetici, dok overflow flag ukazuje na prekoračenje opsega u označenoj aritmetici. [4]

Korišćenje flagova omogućava efikasno donošenje odluka u programu bez dodatnih poređenja i operacija, jer se rezultat instrukcije direktno koristi za uslovno grananje (npr. JZ i JE instrukcije). [3]

Flagovi su implementirani kao pojedinačni bitovi u statusnom registru procesora, dakle predstavljaju sastavni deo mikroarhitekture i ažuriraju se hardverski, paralelno sa izvršavanjem instrukcija u ALU. [4]

1.2 Tehnološke osnove

1.2.1 ALU i generisanje flagova

Aritmetičko-logička jedinica predstavlja deo centralne procesorske jedinice zadužen za izvršavanje aritmetičkih i logičkih operacija nad podacima. ALU prima ulazne operande iz registara, izvršava definisanu operaciju i vraća rezultat nazad u registre ili memoriju. [1]

ALU funkcioniše tako što na osnovu kontrolnih signala iz upravljačke jedinice procesora bira odgovarajuću operaciju i izvršava je nad binarnim podacima. Operacije se izvode u jednom ili više taktova procesora. [2]

Osnovne operacije koje ALU izvršava uključuju aritmetičke operacije, logičke operacije (AND, OR, XOR, NOT), poređenja (compare) i pomeranja bitova (shift/rotate). Rezultat ovih operacija ne utiče samo na izlaz, već i na statusne informacije procesora. [2]

Nakon izvršenja aritmetičke ili logičke operacije, ALU automatski ažurira statusne flagove u statusnom registru procesora. Ovi flagovi nastaju analizom rezultata operacije, bez dodatne softverske intervencije. Ovaj mehanizam omogućava da se rezultat koristi za uslovno izvršavanje narednih instrukcija u programu. [4]

1.2.2 Statusni registri i organizacija flagova

Statusni registri predstavljaju specijalne registre procesora koji sadrže informacije o trenutnom stanju izvršavanja instrukcija. U x86 arhitekturi ovi registri se kroz evoluciju procesora pojavljuju kao FLAGS, EFLAGS i RFLAGS, pri čemu svaki sledeći model proširuje prethodni u skladu sa povećanjem širine procesora. [4]

FLAGS registar predstavlja osnovnu 16-bitnu verziju statusnog registra u ranim x86 procesorima. Sa prelaskom na 32-bitnu arhitekturu je uveden EFLAGS registar, dok se u 64-bitnim procesorima koristi RFLAGS kao proširena verzija. RFLAGS zadržava kompatibilnost sa prethodnim verzijama, ali uvodi dodatne bitove za proširene funkcionalnosti procesora. [4]

Statusni registar je organizovan kao skup pojedinačnih bitova, gde svaki bit predstavlja određeni flag ili kontrolnu informaciju. Ovi bitovi se mogu nezavisno postavljati ili brisati u zavisnosti od rezultata instrukcija ili kontrolnih operacija procesora. Niži bitovi statusnog registra uključuju osnovne aritmetičke flagove, dok viši bitovi sadrže kontrolne informacije vezane za rad procesora i operativnog sistema. [4]

Određeni bitovi u statusnom registru su rezervisani za buduće proširenje arhitekture ili internu upotrebu procesora. Ovi bitovi se ne koriste u programskom kodu i njihova vrednost može biti definisana isključivo od strane hardvera. [4]

Statusni flagovi održavaju rezultat aritmetičkih i logičkih operacija, dok kontrolni flagovi utiču na način izvršavanja instrukcija. Kontrolni flagovi se obično menjaju putem posebnih instrukcija procesora. [4]

1.2.3 Flagovi u assembleru

Flagovi imaju značajnu ulogu u assembler programiranju jer omogućavaju kontrolu toka programa na osnovu rezultata prethodno izvršenih instrukcija. Većina aritmetičkih i logičkih instrukcija automatski menja stanje određenih flagova u statusnom registru procesora. [4]

Instrukcije kao što su ADD, SUB, CMP, INC i DEC menjaju statusne flagove u zavisnosti od rezultata operacije. Na primer, instrukcija ADD može postaviti Carry, Zero, Sign i Overflow flagove. Instrukcija CMP služi za poređenje vrednosti i funkcioniše slično instrukciji SUB, ali bez čuvanja rezultata, pri čemu se ažuriraju samo flagovi. [4]

Logičke instrukcije poput AND, OR i XOR takođe menjaju stanje određenih flagova, posebno Zero i Sign flag. Nakon promene statusnih flagova, moguće je koristiti instrukcije uslovnog skoka koje proveravaju stanje određenog flaga kao što su JZ i JC. [4]

Flagovi imaju važnu ulogu u realizaciji petlji u assembleru jer se najčešće implementiraju pomoću poređenja i uslovnih skokova. [4]

1.2.4 Uticaj flagova na izvršavanje programa

Flagovi imaju važnu ulogu u izvršavanju programa jer omogućavaju procesoru donošenje odluka na osnovu rezultata prethodno izvršenih instrukcija. Njihovo stanje direktno utiče na kontrolu toka programa, obradu grešaka i rad interrupt sistema. [4]

Uslovno grananje predstavlja mehanizam kojim procesor bira različit tok izvršavanja programa u zavisnosti od stanja određenih flagova. Nakon izvršavanja instrukcija poređenja ili aritmetičkih operacija, instrukcije uslovnog skoka proveravaju stanje flagova i odlučuju da li će doći do promene toka programa. [3]

Određeni flagovi mogu ukazivati na greške ili specijalna stanja tokom izvršavanja instrukcija. Pored aritmetičkih grešaka, pojedini flagovi se koriste u sistemskim rutinama i operativnim sistemima za signalizaciju uspešnog ili neuspešnog izvršavanja određenih funkcija. [6]

Interrupt sistem omogućava procesoru da privremeno prekine izvršavanje trenutnog programa radi obrade spoljnog ili unutrašnjeg događaja. Kontrolu nad hardverskim prekidima u x86 arhitekturi vrši interrupt flag (IF). [4]

Flagovi imaju važnu ulogu u radu operativnih sistema, upravljanju prekidima i sinhronizaciji izvršavanja sistemskih rutina. [10]

1.3 Komparativna analiza postojećih rešenja

1.3.1 Flagovi u ARM arhitekturi

ARM arhitektura koristi statusne registre za čuvanje informacija o rezultatima izvršenih instrukcija i za kontrolu rada procesora. Za razliku od x86 arhitekture, gde se koristi jedinstveni FLAGS/EFLAGS/RFLAGS registar, ARM procesori koriste registre kao što su CPSR (Current Program Status Register) i APSR (Application Program Status Register). [5]

U starijim ARM arhitekturama se koristi CPSR registar koji objedinjuje statusne flagove, kontrolne bitove i informacije o režimu rada procesora. CPSR sadrži aritmetičke statusne flagove kao što su Negative, Zero, Carry i Overflow, ali i kontrolne bitove vezane za prekide i režime izvršavanja procesora. [5]

Sa daljim razvojem ARM arhitekture, deo funkcionalnosti CPSR registra je izdvojen u APSR registar, koji sadrži isključivo aplikacione statusne flagove nastale kao rezultat aritmetičkih i logičkih operacija. Na ovaj način omogućeno je jasnije razdvajanje statusnih informacija od sistemskih kontrolnih funkcija. [5]

ARM instrukcije mogu opcionalno da ažuriraju statusne flagove. Instrukcije koje menjaju stanje flagova najčešće imaju sufiks „S“, dok instrukcije bez sufiksa ne menjaju stanje flagova. [5]

Najkarakterističnija osobina ARM arhitekture jeste mogućnost uslovnog izvršavanja instrukcija. Na ovaj način se smanjuje potreba za čestim grananjem programa i povećava efikasnost izvršavanja instrukcija. [5]

ARM arhitektura podržava različite uslove izvršavanja na osnovu statusnih flagova. Neki od najčešće korišćenih uslova su:

Oznaka	Značenje	Proveravani uslov
EQ	Jednako	$Z = 1$
NE	Nejednako	$Z = 0$
CS/HS	Prenos postavljen / Veće ili isto	$C = 1$
CC/LO	Prenos nije postavljen / Manje	$C = 0$
MI	Negativno	$N = 1$
PL	Pozitivno	$N = 0$

Tabela 2 – Česti uslovi izvršavanja instrukcija u ARM arhitekturi [5]

1.3.2 RISC arhitekture i alternativni pristupi

Za razliku od x86 arhitekture, mnoge RISC arhitekture ne koriste poseban statusni registar za čuvanje aritmetičkih flagova nakon svake operacije. Umesto toga, poređenja i odluke o grananju se najčešće realizuju direktnim poređenjem registara. [3]

RISC-V i MIPS predstavljaju tipične primere arhitekture koje izbegavaju klasičan koncept statusnog registra sa flagovima kao što su Zero, Carry ili Overflow. Ovakav pristup pojednostavljuje dizajn procesora, smanjuje međuzavisnost instrukcija i omogućava efikasnije izvršavanje instrukcija u organizaciji procesora. [3][7]

Kod MIPS arhitekture, grananje se realizuje instrukcijama kao što su BEQ (Branch if equal) i BNE (Branch not equal), koje direktno porede sadržaj dva registra bez korišćenja statusnog registra. [3]

RISC-V arhitektura koristi sličan princip, gde instrukcije grananja direktno proveravaju odnos između registara, bez oslanjanja na posebne statusne flagove. [7]

1.3.3 Poređenje arhitektura

Arhitektura	Statusni registar	Način grananja	Karakterističan pristup
x86	FLAGS / EFLAGS / RFLAGS	Grananje preko statusnih flagova	Automatsko ažuriranje flagova nakon ALU operacija
ARM	CPSR / APSR	Grananje i uslovno izvršavanje preko flagova	Opciono ažuriranje flagova instrukcijama
MIPS	Nema statusne flagove za grananje	Direktno poređenje registara	Grananje bez statusnih flagova
RISC-V	Bez statusnog registra za ALU flagove	Direktno poređenje registara	Smanjena međuzavisnost instrukcija i jednostavniji dizajn

Tabela 3 – Tabelarno poređenje arhitektura [3][4][5][7]

U tabeli se nalazi poređenje pomenutih procesorskih arhitektura prema načinu organizacije statusnih registara i realizacije uslovnog grananja. Arhitektura x86 i ARM koriste statusne flagove za donošenje odluka, dok MIPS i RISC-V koriste direktno poređenje registara bez klasičnog statusnog registra za aritmetičke flagove. Ovakve razlike predstavljaju posledicu različitih pristupa u dizajnu procesora između CISC i RISC arhitektura. [3][4][5][7]

1.4 Primeri primene

1.4.1 Kontrola toka programa

Uslovno grananje u assembleru najčešće se realizuje kombinacijom instrukcija za poređenje i instrukcija uslovnog skoka. Instrukcija CMP poredi vrednosti operanada tako što interno izvršava operaciju oduzimanja bez čuvanja rezultata, pri čemu ažurira statusne flagove. Nakon toga instrukcije kao što su JE i JNE proveravaju stanje Zero flaga. [4]

Na primer, ukoliko su vrednosti registara jednake, Zero flag dobija vrednost 1 i izvršava se instrukcija JE.

```
.code

CMP AX,BX

JE jednako
JMP nejednako

jednako:
MOV DX,1
JMP kraj

nejednako:
MOV DX,0

kraj:
```

Listing 1 – primer asemblerskog koda koji prikazuje kontrolu toka programa. [6]

U prikazanom listingu dolazi do skoka na labelu „jednako“ ukoliko su registri AX i BX jednaki, dok u suprotnom slučaju dolazi do skoka na labelu „nejednako“.

Na ovaj način se realizuje logika ekvivalentna if-else grananju u programskim jezicima višeg nivoa. [4]

Flagovi imaju važnu ulogu u realizaciji petlji u assembler programiranju. Petlje se često implementiraju pomoću instrukcija koje menjaju registre brojača i instrukcija uslovnog skoka koje proveravaju stanje statusnih flagova. [3]

```
.code

MOV CX,10

petlja:
DEC CX
```

JNZ petlja

MOV DX,1

Listing 2 – primer asemblerskog koda koji predstavlja implementaciju koncepta petlje. [6]

U listingu 2 se izvršava petlja koja dekrementuje registar CX sve dok vrednost u CX registru ne postane jednaka nuli. Kada se petlja završi, program skladišti broj 1 u DX registar.

Strukture višestrukog grananja, ekvivalentne switch naredbama u jezicima višeg nivoa, mogu se realizovati korišćenjem više uzastopnih instrukcija poređenja i uslovnih skokova. Instrukcije uslovnog skoka proveravaju stanje odgovarajućih statusnih flagova u statusnom registru procesora. [3]

.code

CMP AX,1

JE opcija1

CMP AX, 2

JE opcija2

JMP default

opcija1:

MOV DX,1

opcija2:

MOV DX,2

default:

MOV DX,3

Listing 3 – primer asemblerskog koda koji predstavlja implementaciju strukture višestrukog grananja. [6]

U listingu 3 dolazi do sekvencijalne provere registra AX kroz više mogućih opcija dok se ne stigne do default opcije. U slučaju da se ispuni uslov bilo koje opcije, program izvršava skok na određenu labelu i upisuje vrednost u registar DX.

Na ovaj način se realizuje višestruko grananje, pri čemu svaka CMP instrukcija menja stanje statusnih flagova koji određuju da li će doći do uslovnog skoka. [4]

1.4.2 Aritmetičke operacije

Flagovi imaju značajnu ulogu u aritmetičkim operacijama jer omogućavaju procesoru da detektuje specijalna stanja kao što su prenos, pozajmica i prekoračenje opsega rezultata. [4]

Carry flag (CF) koristi se prvenstveno kod operacija nad neoznačenim (unsigned) brojevima. Ovaj flag dobija vrednost 1 ukoliko prilikom sabiranja dođe do prenosa iz najvišeg bita rezultata ili ukoliko kod oduzimanja dođe do pozajmice. [4]

```
.code  
MOV AL,255  
ADD AL,1
```

Listing 4 – primer postavljanja Carry flag-a nakon prekoračenja opsega neoznačenog broja. [6]

U prikazanom primeru dolazi do sabiranja vrednosti 255 i 1 u 8-bitnom registru AL. Pošto maksimalna vrednost neoznačenog 8-bitnog broja iznosi 255, rezultat prelazi dozvoljeni opseg registra i carry flag dobija vrednost 1. [4]

Overflow flag (OF) koristi se kod označenih (signed) brojeva i signalizira prekoračenje dozvoljenog opsega rezultata u aritmetici sa predznakom. Overflow nastaje kada rezultat operacije nije moguće predstaviti sa raspoloživim brojem bitova. [4]

```
.code  
MOV AL,127  
ADD AL,1
```

Listing 5 – primer postavljanja Overflow flag-a nakon prekoračenja opsega označenog broja.
[6]

U ovom primeru registru AL se dodeljuje najveća pozitivna vrednost označenog 8-bitnog broja. Nakon sabiranja sa brojem 1 dolazi do prekoračenja opsega označenih brojeva, zbog čega Overflow flag dobija vrednost 1. [4]

Razlika između Carry i Overflow flagova predstavlja važan aspekt procesorske aritmetike. Carry flag ukazuje na prekoračenje kod neoznačenih brojeva, dok Overflow flag označava grešku u interpretaciji označenih brojeva. Iako obe situacije predstavljaju prekoračenje opsega registra, njihovo značenje zavisi od načina interpretacije podataka. [3][4]

1.4.3 Operativni sistemi

Statusni i kontrolni flagovi predstavljaju važan deo komunikacije između procesora i operativnog sistema. [10]

Interrupt sistem omogućava procesoru da privremeno prekine izvršavanje trenutnog programa radi obrade spoljnog ili unutrašnjeg događaja. Kontrolu nad hardverskim prekidima u x86 arhitekturi vrši Interrupt flag (IF), koji se nalazi u statusnom registru procesora. [4]

Kada interrupt flag ima vrednost 1, procesor prihvata maskabilne hardverske prekide. Ukoliko se Interrupt flag postavi na 0, hardverski prekidi se privremeno onemogućavaju. Operativni sistemi često privremeno isključuju prekide tokom izvršavanja kritičnih sekcija kako bi sprečili nekonzistentno stanje podataka. [10]

Savremeni operativni sistemi koriste različite režime rada procesora kako bi odvojili izvršavanje korisničkih aplikacija od sistemskih funkcija. Kernel režim omogućava izvršavanje privilegovanih instrukcija i direktan pristup hardverskim resursima računara. [10]

Prilikom izvršavanja sistemskih rutina i obrade prekida, procesor ulazi u privilegovani režim rada i koristi statusni registar za očuvanje kontrolnih informacija o trenutnom stanju izvršavanja programa. Pojedini flagovi određuju način obrade prekida i ponašanje procesora tokom izvršavanja sistemskog koda. [4]

Promena konteksta (context switch) predstavlja postupak kojim operativni sistem prekida izvršavanje jednog procesa i nastavlja izvršavanje drugog procesa. Tokom ove operacije neophodno je sačuvati trenutno stanje procesora kako bi se kasnije moglo nastaviti izvršavanje prekinutog procesa. [10]

1.4.4 Embedded sistemi i mikrokontroleri

Zbog ograničenih hardverskih resursa i potrebe za brzom reakcijom na spoljne događaje, statusni registri i flagovi predstavljaju važan deo upravljanja mikrokontrolerima. [8]

ARM Cortex-M mikrokontroleri koriste APSR za čuvanje statusnih flagova nastalih kao rezultat aritmetičkih i logičkih operacija. U embedded sistemima baziranim na ARM Cortex-M arhitekturi flagovi se često koriste u obradi prekida, upravljanju periferijama i realizaciji real-time zadataka. [5]

AVR i PIC mikrokontroleri takođe koriste statusne registre koji sadrže flagove neophodne za upravljanje izvršavanjem instrukcija. Kod AVR arhitekture statusni registar SREG sadrži flagove kao što su Carry, Zero, Negative i Overflow. [8]

PIC mikrokontroleri koriste sličan pristup kroz STATUS registar koji sadrži informacije o rezultatima prethodno izvršenih operacija. Statusni flagovi omogućavaju jednostavnu realizaciju upravljačke logike u sistemima sa ograničenim resursima. [8]

Embedded sistemi često rade u realnom vremenu i moraju brzo reagovati na promene spoljašnjeg okruženja. Mikrokontroleri u takvim sistemima koriste prekide i statusne flagove kako bi efikasno obradili događaje generisane od strane senzora, komunikacionih modula i drugih perifernih uređaja. [5]

1.4.5 Debugovanje i reverzni inženjering

Debuggeri predstavljaju alate koji omogućavaju programerima i analitičarima praćenje rada programa tokom izvršavanja. Tokom debugovanja moguće je pratiti sadržaj registara procesora, memorije i statusnih flagova nakon svake izvršene instrukcije. [9]

Debuggeri omogućavaju postavljanje prekidnih tačaka (breakpoints), pri čemu se izvršavanje programa privremeno zaustavlja na određenoj instrukciji. Nakon zaustavljanja programa moguće je analizirati stanje registara i statusnih flagova kako bi se utvrdilo ponašanje programa u određenom trenutku izvršavanja. [9]

Izvršavanje po koraku (single-step execution) predstavlja tehniku debugovanja pri kojoj procesor izvršava jednu po jednu instrukciju uz mogućnost analize stanja sistema nakon svakog koraka. Ovakav način rada omogućava detaljno praćenje promena registara, memorije i statusnih flagova. [9]

U x86 arhitekturi za realizaciju izvršavanja po koraku koristi se Trap flag (TF), koji se nalazi u statusnom registru procesora. Kada je Trap flag postavljen na vrednost 1, procesor nakon izvršavanja svake instrukcije generiše poseban debug prekid. Na ovaj način debugger dobija mogućnost kontrole izvršavanja programa instrukciju po instrukciju. [4]

Trap flag predstavlja važan mehanizam u analiziranju izvršavanja programa, otkrivanju grešaka i reverznom inženjeringu softvera, jer omogućava detaljan uvid u ponašanje procesora. [4][9]

2. Praktični deo

2.1 Namena softvera i korisničko uputstvo

Razvijeni softver predstavlja jednostavan simulator procesora i izvršavanja asemblerskih instrukcija sa podrškom za procesorske flagove. Cilj programa je demonstracija rada osnovnih registara, instrukcija i statusnih flagova procesora kroz simulaciju izvršavanja instrukcija nad 8-bitnom arhitekturom.

Program omogućava učitavanje pseudo-asemblerskog programa iz tekstualne datoteke, praćenje promena registara, statusnih flagova i promenljivih, uslovne i bezuslovne skokove, kao i generisanje log fajla sa kompletnim tokom izvršavanja programa.

Softver je prvenstveno edukativnog karaktera i namenjen je za lakše razumevanje načina rada procesorskih flagova kao što su Carry Flag, Sign Flag, Overflow Flag i Zero Flag.

Zbog načina na koji radno okruženje pristupa direktorijumu, čitanje fajlova je moguće isključivo preko generisanog .exe fajla. Iz tog razloga, program se prvo kompajlira u radnom okruženju nakon čega se pokreće putem izvršne .exe datoteke, kako bi program koristio ispravnu putanju za pristup datotekama.

Prilikom pokretanja programa se korisniku prikazuje lista dostupnih .txt fajlova iz „assemblerski_programi“ direktorijuma. Korisnik dobija mogućnost da odabere jedan od prikazanih fajlova. Posle odabira, program se automatski učitava i izvršava. Tokom izvršavanja se svaki korak i greške upisuju u log fajl koji se nalazi u „logovi“ direktorijumu. U slučaju greške se program zaustavlja na koraku gde je greška pronađena, prikazuje je na ekranu i upisuje u log fajl.

```
Dostupne datoteke:
1. program1.txt
2. program_petlja.txt
3. test.txt
Unesite broj datoteke koju zelite da ucitate:
```

Slika 1 – Lista dostupnih .txt datoteka u direktorijumu „assemblerski_programi“

```
Dostupne datoteke:
1. program1.txt
2. program_petlja.txt
3. test.txt
Unesite broj datoteke koju zelite da ucitate: 1
Odabrana datoteka: assembler_programi\program1.txt
Obrada linije u CODE segmentu: MOV AL,1

Obrada linije u CODE segmentu: INC AL

Obrada linije u CODE segmentu: INC AL

Obrada linije u CODE segmentu: INC AL
```

Slika 2 – Odabir datoteke i njeno učitavanje u program

```
--- Korak 1 ---  
Instrukcija: 0, Operand1: AL, Operand2: 1  
  
Registri:  
AL:1, BL:0, CL:0, DL:0  
  
Flagovi:  
CF:0, ZF:0, OF:0, SF:0  
  
Promenljive:  
  
--- Kraj Koraka 1 ---
```

Slika 3 – Primer ispisa koraka operacije

Pošto program razume pseudo-asmblerski kod, postoje brojna pravila koja se razlikuju od oficijalnog MASM x86 jezika. Program prepoznaje segment podataka i segment koda. Svaki program mora imati segment koda, dok je segment podataka opcionalan ukoliko ne postoje promenljive koje se koriste. Nazivi segmenta moraju biti napisani malim slovima.

Promenljive u segmentu podataka se definišu bez ključne reči „db“ (define byte), tako da njihovu definiciju čini samo naziv i vrednost, između kojih mora postojati razmak. U celom programu podržane su isključivo vrednosti tipa int. Naziv promenljive se mora nalaziti na početku linije segmenta podataka. Imena promenljivih ne smeju biti ista kao imena registara, instrukcija i labela. Imena promenljive ne smeju sadržati razmake.

Podržani registri su AL,BL,CL i DL. Svi registri moraju biti napisani velikim slovima.

Podržane instrukcije su MOV,ADD,SUB,CMP,INC,DEC,JMP i JE. Sve instrukcije moraju biti napisane velikim slovima. Naziv instrukcije se mora nalaziti na početku linije kodnog segmenta. Između instrukcije i operandada mora postojati razmak.

Operandi mogu biti registri, promenljive i numeričke vrednosti int tipa. Labela su validan operand samo u instrukcijama uslovnog i bezuslovnog skoka. U instrukcijama sa više operandada, između dva operandada mora postojati zarez. Nazivi operandada mogu biti maksimalne dužine 15 karaktera.

Labela se pišu u posebnoj liniji iznad instrukcija na koju pokazuju. Labela ne sme biti u istoj liniji kao i instrukcija. Labela se definiše kao naziv sa dvotačkom na kraju. Tokom korišćenja labela kao operand se ne piše dvotačka.

Program se automatski prekida ako je korišćen operand ili instrukcija koja ne postoji. Korišćenje netačnog broja operandada za određenu instrukciju terminira program.

```

.data

pocetak 0

cilj 100

stigao_do_kraja 0


.code

MOV AL, pocetak


petlja:

INC AL

CMP AL, cilj

JE kraj

JMP petlja


kraj:

MOV stigao_do_kraja, 1

```

Listing 6 – primer pravilno napisanog pseudo-asmblerskog koda

2.2 Kratak opis primenjenih alata i tehnologija u implementaciji rešenja

Za implementaciju softvera je korišćen programski jezik C. Program je razvijan u razvojnom okruženju Visual Studio Code, dok je za kompajliranje koda korišćen GCC kompajler. Aplikacija je razvijana i testirana na operativnom sistemu Windows 10.

Tokom implementacije korišćene su standardne biblioteke iz C jezika:

- `stdio.h` – za ulazno-izlazne operacije i rad sa fajlovima
- `stdlib.h` – za osnovne sistemske funkcije i upravljanje memorijom
- `string.h` – za obradu stringova
- `ctype.h` – za proveru i obradu karaktera
- `windows.h` – za čitanje imena fajlova iz direktorijuma
- `time.h` – za generisanje datuma u imenu log datoteka

Pseudo-asmblerski programi se čuvaju u .txt datotekama, dok se informacije o izvršavanju programa i eventualnim greškama automatski upisuju u log fajlove radi lakšeg praćenja rada simulatora.

2.3 Opis implementacije

Važno je napomenuti da je softver zasnovan na neoznačenoj 8-bitnoj arhitekturi, dok se označena 8-bitna reprezentacija koristi isključivo prilikom interpretacije vrednosti Sign i Overflow flagova.

Softver sadrži strukture za promenljive, labele, operande, instrukcije, datoteke i samu centralnu procesorsku jedinicu. Definisani su makroi za 8-bitnu arhitekturu kao i enumeracije za dozvoljene registre, flagove, instrukcije, vrste operanada i segmente.

```
//Definisanje makroa za 8bit arhitekturu
#define MASKA_8bit 0xFF
#define ZNAKOVNI_8bit 0x80

// Definisanje enumeracija za registre, flagove, instrukcije i segmente
typedef enum {AL,BL,CL,DL, REGISTRAR_NEISPRAVAN} ImeRegistra;
typedef enum {CF,ZF,OF,SF, FLAG_NEISPRAVAN} ImeFlaga;
typedef enum {MOV,ADD,SUB,CMP,INC,DEC,JMP,JE, INSTRUKCIJA_NEISPRAVNA} ImeInstrukcije;
typedef enum {SEGMENT_NULL,SEGMENT_DATA,SEGMENT_CODE} Segment;
typedef enum {REGISTAR,PROMENLJIVA,VREDNOST,LABELA, OPERAND_NEISPRAVAN} TipOperanda;
```

Slika 4 – definicije makroa i enumeracija

```
// Definisanje struktura za promenljive, labele, operande, instrukcije, CPU i datoteke
typedef struct Promenljiva {
    char ime_promenljive[32];
    int vrednost_promenljive;
} Promenljiva;

typedef struct Labela {
    char ime_labela[32];
    struct Instrukcija *instrukcija_labela;
} Labela;

typedef struct Operand {
    TipOperanda tip_operanda;
    int vrednost_operanda; //Koristi se samo ako je operand tipa VREDNOST
    char ime_labela[32]; //Koristi se samo ako je operand tipa LABELA
    ImeRegistra registar;
    Promenljiva *promenljiva;
} Operand;

typedef struct Instrukcija {
    ImeInstrukcije ime_instrukcije;
    char operand1[16];
    char operand2[16];
    struct Instrukcija *sledeca_instrukcija;
} Instrukcija;

typedef struct CPU {
    int registri[4];
    int flagovi[4];

    Promenljiva promenljive[MAX_PROMENLJIVIH];
    Labela labela[MAX_LABELA];
} CPU;

typedef struct datoteka {
    char naziv[MAX_PUTANJA_FAJLA];
} datoteka;
```

Slika 5 – strukture u programu

```

Instrukcija* kreirajInstrukciju(char *ime_instrukcije, char *operand1, char *operand2) {

    // Alokacija memorije za novu instrukciju
    Instrukcija *nova_instrukcija = (Instrukcija*)malloc(sizeof(Instrukcija));
    if (nova_instrukcija == NULL) {
        printf("Greska pri alokaciji memorije za novu instrukciju.\n");
        exit(EXIT_FAILURE);
    }

    // Parsiranje instrukcije i provera validnosti
    int parsirana_instrukcija = parsirajInstrukciju(ime_instrukcije);
    if (parsirana_instrukcija == INSTRUKCIJA_NEISPRAVNA) {
        prijaviGresku("Nevalidna instrukcija.\n");
        free(nova_instrukcija);
        exit(EXIT_FAILURE);
    }
    nova_instrukcija->ime_instrukcije = parsirana_instrukcija;

    // Kopiranje operandi i osiguravanje da su null-terminirani
    strncpy(nova_instrukcija->operand1, operand1, sizeof(nova_instrukcija->operand1) - 1);
    nova_instrukcija->operand1[sizeof(nova_instrukcija->operand1) - 1] = '\0';

    strncpy(nova_instrukcija->operand2, operand2, sizeof(nova_instrukcija->operand2) - 1);
    nova_instrukcija->operand2[sizeof(nova_instrukcija->operand2) - 1] = '\0';

    // Inicijalizacija pokazivača na sledeću instrukciju
    nova_instrukcija->sledeca_instrukcija = NULL;

    return nova_instrukcija;
}

```

Slika 6 – funkcija „kreirajInstrukciju“

Struktura za instrukcije je sastavni deo spregnute liste koja je centralni deo programa.

Funkcija „kreirajInstrukciju“ prima podatke pročitane iz linije datoteke, alocira memoriju za podatke koje unosi u strukturu posle validacije instrukcije. Operandi su kopirani na bezbedan način i terminirani.

```

void dodajInstrukciju(Instrukcija *nova_instrukcija) {
    if (glava_liste_instrukcija == NULL) {
        glava_liste_instrukcija = nova_instrukcija;
        rep_liste_instrukcija = nova_instrukcija;
    } else {
        rep_liste_instrukcija->sledeca_instrukcija = nova_instrukcija;
        rep_liste_instrukcija = nova_instrukcija;
    }
}

void oslobodiInstrukcije() {
    while (glava_liste_instrukcija != NULL) {
        Instrukcija *temp = glava_liste_instrukcija;
        glava_liste_instrukcija = glava_liste_instrukcija->sledeca_instrukcija;
        free(temp);
    }
}

```

Slika 7 – funkcije „dodajInstrukciju“ i „oslobodiInstrukcije“

Funkcije dodajInstrukciju i oslobodiInstrukcije su ključne za rad sa ovom strukturom u kontekstu spregnute liste. **Funkcija dodajInstrukciju** dodaje strukturu na kraj liste. Ukoliko je lista prazna, određena struktura postaje glava i rep liste. **Funkcija oslobodiInstrukcije** briše sve učitane instrukcije iz liste.

```

int UcitajInstrukcijeIzFajla(char *putanja_fajla, CPU *cpu) {
    //Kontrolne promenljive
    char poslednja_labela[32];
    int labela_trazi_sledecu_instrukciju = 0;
    Segment trenutni_segment = SEGMENT_NULL;

    // Otvaranje fajla za citanje
    FILE *fajl = fopen(putanja_fajla, "r");
    if (fajl == NULL) {
        printf("Greska pri otvaranju fajla.\n");
        exit(EXIT_FAILURE);
    }

    int greskacitanja = 0;

    // Citanje linija iz fajla
    char linija[MAX_LINIJA_FAJLA];
    while (fgets(linija, sizeof(linija), fajl) != NULL) {

        // Promenljive za parsiranje instrukcije i operandi
        char ime_instrukcije[16];
        char operand1[16];
        char operand2[16];

        operand1[0] = '\0';
        operand2[0] = '\0';

        //Ako je linija prazna ili sadrzi samo razmake, preskacemo je
        char trim_linija[MAX_LINIJA_FAJLA];
        strcpy(trim_linija, linija);
        trim(trim_linija);

        if (strlen(trim_linija) == 0) {
            continue;
        }

        // Provera da li linija sadrzi segment
        if (strcmp(trim_linija, ".data") == 0) {
            trenutni_segment = SEGMENT_DATA;
            continue;
        } else if (strcmp(trim_linija, ".code") == 0) {
            trenutni_segment = SEGMENT_CODE;
            continue;
        }
    }
}

```

Slika 8 – prvi deo funkcije „UcitajInstrukcijeIzFajla“

```

//Segmentovanje
if (trenutni_segment == SEGMENT_NULL) {

    continue;

} else if (trenutni_segment == SEGMENT_DATA) {

    // Provera da li linija sadrzi promenljivu
    char ime_promenljive[32];
    int vrednost_promenljive;
    int rezultat = sscanf(linija, "%31s %d", ime_promenljive, &vrednost_promenljive);
    if (rezultat != 2) {

        char greska[128];
        sprintf(greska, "Nevalidan format promenljive u fajlu: %s", linija);
        prijaviGresku(greska);

        greskacitanja = 1;
        break;

    }

    trim(ime_promenljive);
    vrednost_promenljive = vrednost_promenljive & MASKA_8bit;

    // Dodavanje promenljive u CPU
    Promenljiva *nova_promenljiva = kreirajPromenljivu(ime_promenljive, vrednost_promenljive);

    int dodato = 0;
    for (int i = 0; i < MAX_PROMENLJIVIH; i++) {
        if (strcmp(cpu->promenljive[i].ime_promenljive, ime_promenljive) == 0) {
            printf("Promenljiva sa imenom '%s' vec postoji. Unosenje nove vrednosti na rezervisano mesto.\n", ime_promenljive);
            cpu->promenljive[i] = *nova_promenljiva;
            free(nova_promenljiva);
            dodato = 1;
            break;
        }

        if (cpu->promenljive[i].ime_promenljive[0] == '\0') { // Pronadji prvo slobodno mesto
            cpu->promenljive[i] = *nova_promenljiva;
            free(nova_promenljiva);
            dodato = 1;
            break;
        }
    }
}

```

Slika 9 – drugi deo funkcije „UcitajInstrukcijeIzFajla“

```

    if (!dodato) {
        printf("Nema mesta za novu promenljivu '%s'.\n", ime_promenljive);
        free(nova_promenljiva);
    }

} else if (trenutni_segment == SEGMENT_CODE) {

    printf("Obrada linije u CODE segmentu: %s\n", linija);

    // Provera da li linija sadrzi labelu
    if (detektujLabelu(linija, poslednja_labela)) {
        // Ako linija sadrzi labelu, preskacemo je
        labela_trazi_sledecu_instrukciju = 1;
        continue;
    }

    // Parsiranje instrukcije i provera validnosti
    int parsirana_instrukcija;
    sscanf(linija, "%15s", ime_instrukcije);
    trim(ime_instrukcije);
    parsirana_instrukcija = parsirajInstrukciju(ime_instrukcije);
    if (parsirana_instrukcija == INSTRUKCIJA_NEISPRAVNA) {

        char greska[128];
        sprintf(greska, "Nevalidna instrukcija u fajlu: %s\n", ime_instrukcije);
        prijaviGresku(greska);

        greskacitanja = 1;
        break;
    }
}

```

Slika 10 – treći deo funkcije „UcitajInstrukcijeIzFajla“

```

switch(parsirana_instrukcija) {
    case MOV:
    case ADD:
    case SUB:
    case CMP:
        int rezultat2op = sscanf(linija, "%*s %15[^,], %15s", operand1, operand2);

        if (rezultat2op != 2) {
            char greska[128];
            sprintf(greska, "Nevalidan format instrukcije u fajlu: %s\n", linija);
            prijaviGresku(greska);

            greskacitanja = 1;
            break;
        }

        trim(operand1);
        trim(operand2);

        break;
    case INC:
    case DEC:
    case JMP:
    case JE:
        int rezultat1op = sscanf(linija, "%*s %15s", operand1);

        if (rezultat1op != 1) {
            char greska[128];
            sprintf(greska, "Nevalidan format instrukcije u fajlu: %s\n", linija);
            prijaviGresku(greska);
            greskacitanja = 1;
            break;
        }

        trim(operand1);
        operand2[0] = '\0'; // Postavljanje drugog operanda na prazan string

        break;
    default:
        char greska[128];
        sprintf(greska, "Nevalidna instrukcija u fajlu: %s\n", ime_instrukcije);
        prijaviGresku(greska);

        greskacitanja = 1;

```

Slika 11 – četvrti deo funkcije „UcitajInstrukcijeIzFajla“

```

        default:
            char greska[128];
            sprintf(greska, "Nevalidna instrukcija u fajlu: %s\n", ime_instrukcije);
            prijaviGresku(greska);

            greskacitanja = 1;
            break;
    }

    Instrukcija *nova_instrukcija = kreirajInstrukciju(ime_instrukcije, operand1, operand2);

    //Ako je pre ove linije bila detektovana labela, dodajemo instrukciju u tu labelu
    if (labela_trazi_sledecu_instrukciju) {
        //Dodati instrukciju u labelu
        Labela *nova_labela = kreirajLabelu(poslednja_labela, nova_instrukcija);
        dodajLabelu(nova_labela, cpu);

        labela_trazi_sledecu_instrukciju = 0;
    }

    dodajInstrukciju(nova_instrukcija);
}

}

fclose(fajl);

if (greskacitanja) {
    return 0;
}
return 1;
}

```

Slika 12 – peti deo funkcije „UcitajInstrukcijeIzFajla“

Funkcija „UcitajInstrukcijeIzFajla“ je od ključnog značaja jer od nje zavise sve informacije koje se koriste u izvršavanju programa. Prvenstveno se otvara izabrana datoteka u direktorijumu „assembler_programi“. Iz datoteke se čitaju informacije liniju po liniju, svrstavaju se u segment podataka ili segment koda. U segmentu podataka se čitaju promenljive i čuvaju u CPU strukturu pomoću strukture „Promenljiva“. U segmentu koda se čitaju instrukcije, dele se na mnemonik i operande. Mnemonik i operandi se čuvaju u strukturu „Instrukcija“, nakon čega struktura postaje deo spregnute liste. Ukoliko se u kodnom segmentu nalazi labela, traži se sledeća validna instrukcija i povezuje se na labelu uz pomoć pokazivača na tu instrukciju.

```

void izvršiInstrukcije(CPU *cpu) {

    int korak = 0;
    Instrukcija *trenutna = glava_liste_instrukcija;

    while (trenutna != NULL) {
        korak++;

        Operand Operand1 = parsirajOperand(cpu, trenutna->operand1);
        Operand Operand2 = parsirajOperand(cpu, trenutna->operand2);

        int treba_operand2 = 1;
        switch(trenutna->ime_instrukcije) {
            case INC:
            case DEC:
            case JMP:
            case JE:
                treba_operand2 = 0;
                break;
        }

        if (Operand1.tip_operanda == OPERAND_NEISPRAVAN || (Operand2.tip_operanda == OPERAND_NEISPRAVAN && treba_operand2)) {
            prijaviGresku("Operand ne postoji.\n");
            break;
        }

        int Vrednost_Operand1 = parsirajVrednost(cpu, Operand1) & MASKA_8bit;
        int Vrednost_Operand2 = parsirajVrednost(cpu, Operand2) & MASKA_8bit;

        //Labela samo za JMP instrukciju
        if (Operand1.tip_operanda == LABELA && trenutna->ime_instrukcije != JMP && trenutna->ime_instrukcije != JE) {

            char greska[128];
            sprintf(greska, "Nevalidan operand: Labela '%s' se ne moze koristiti kao operand osim u JMP instrukcijama.\n", Operand1.ime_labela);
            prijaviGresku(greska);

            trenutna = trenutna->sledeca_instrukcija;
            break;
        }

        Instrukcija *sledeca_instrukcija = trenutna->sledeca_instrukcija;
    }
}

```

Slika 13 – prvi deo funkcije „izvršiInstrukcije“

```

//Izvršavanje instrukcije na osnovu njenog imena
switch(trenutna->ime_instrukcije) {
    case MOV: {
        upisiVrednost(cpu, Operand1, Vrednost_Operand2);
        break;
    }
    case ADD: {
        int inicijalni = Vrednost_Operand1 + Vrednost_Operand2;
        int rezultat = inicijalni & MASKA_8bit; // Ograničavanje rezultata na 8 bitova

        upisiVrednost(cpu, Operand1, rezultat);
        upisiFlagove(cpu, trenutna->ime_instrukcije, inicijalni, rezultat, Vrednost_Operand1, Vrednost_Operand2);

        break;
    }
    case SUB: {
        int inicijalni = Vrednost_Operand1 - Vrednost_Operand2;
        int rezultat = inicijalni & MASKA_8bit; // Ograničavanje rezultata na 8 bitova

        upisiVrednost(cpu, Operand1, rezultat);
        upisiFlagove(cpu, trenutna->ime_instrukcije, inicijalni, rezultat, Vrednost_Operand1, Vrednost_Operand2);

        break;
    }
    case CMP: {
        int inicijalni = Vrednost_Operand1 - Vrednost_Operand2;
        int rezultat = inicijalni & MASKA_8bit; // Ograničavanje rezultata na 8 bitova

        upisiFlagove(cpu, trenutna->ime_instrukcije, inicijalni, rezultat, Vrednost_Operand1, Vrednost_Operand2);

        break;
    }
    case INC: {
        int inicijalni = Vrednost_Operand1 + 1;
        int rezultat = inicijalni & MASKA_8bit; // Ograničavanje rezultata na 8 bitova

        upisiVrednost(cpu, Operand1, rezultat);
        upisiFlagove(cpu, trenutna->ime_instrukcije, inicijalni, rezultat, Vrednost_Operand1, Vrednost_Operand2);

        break;
    }
}

```

Slika 14 – drugi deo funkcije „izvrsiInstrukcije“

```

    }
    case DEC: {
        int inicijalni = Vrednost_Operand1 - 1;
        int rezultat = inicijalni & MASKA_8bit; // Ograničavanje rezultata na 8 bitova

        upisiVrednost(cpu, Operand1, rezultat);
        upisiFlagove(cpu, trenutna->ime_instrukcije, inicijalni, rezultat, Vrednost_Operand1, Vrednost_Operand2);

        break;
    }
    case JMP: {
        // Implementacija skoka

        Labela *labela = pronadjiLabelu(cpu, Operand1.ime_labela);
        if (labela != NULL) {
            sledeca_instrukcija = labela->instrukcija_labela;
        } else {
            char greska[128];
            sprintf(greska, "Labela '%s' nije pronadjena za JMP instrukciju.\n", Operand1.ime_labela);
            prijaviGresku(greska);
        }

        break;
    }
    case JE: {
        if (cpu->flagovi[ZF] == 1) {
            Labela *labela = pronadjiLabelu(cpu, Operand1.ime_labela);

            if (labela != NULL) {
                sledeca_instrukcija = labela->instrukcija_labela;
            } else {
                char greska[128];
                sprintf(greska, "Labela '%s' nije pronadjena za JE instrukciju.\n", Operand1.ime_labela);
                prijaviGresku(greska);
            }
        }
        break;
    }
}

```

Slika 15 – treći deo funkcije „izvrsiInstrukcije“

```

    }
    default: {
        prijaviGresku("Nevalidna instrukcija za izvorsavanje.\n");
        break;
    }
}

ispisCPU(cpu, korak, trenutna);
upis_Log(cpu, korak, trenutna);

trenutna = sledeca_instrukcija;
}
}

```

Slika 16 – četvrti deo funkcije „izvrsiInstrukcije“

Funkcija „izvrsiInstrukcije“ prolazi kroz spregnutu listu i izvršava učitane instrukcije jednu po jednu. Tokom izvršavanja prati broj instrukcija i takođe upisuje podatke o statusu centralne procesorske jedinice u log datoteku i ispisuje ih na ekran. Status centralne procesorske jedinice se menja prema instrukciji i njenim operandima. Proverava se da li su tipovi operandi dozvoljeni za određene instrukcije. Funkcija podržava rad sa labelama i skokovima.

```

1 void upis_Log(CPU *cpu, int korak, Instrukcija *trenutna_instrukcija) {
2     FILE *log_fajl = fopen(naziv_log_fajla, "a");
3     if (log_fajl == NULL) {
4         printf("Greska pri otvaranju log fajla za upis.\n");
5         return;
6     }
7     fprintf(log_fajl, "\n--- Korak %d ---\n", korak);
8     fprintf(log_fajl, "Instrukcija: %d, Operand1: %s, Operand2: %s\n", trenutna_instrukcija->ime_instrukcije, trenutna_instrukcija->operand1, trenutna_instrukcija->operand2);
9     fprintf(log_fajl, "Registri: AL:%d, BL:%d, CL:%d, DL:%d\n", cpu->registri[AL], cpu->registri[BL], cpu->registri[CL], cpu->registri[DL]);
10    fprintf(log_fajl, "Flagovi: CF:%d, ZF:%d, OF:%d, SF:%d\n", cpu->flagovi[CF], cpu->flagovi[ZF], cpu->flagovi[OF], cpu->flagovi[SF]);
11    fprintf(log_fajl, "Promenljive: ");
12
13    for (int i = 0; i < MAX_PROMENLJIVIH; i++) {
14        if (cpu->promenljive[i].ime_promenljive[0] != '\0') {
15            fprintf(log_fajl, "%s: %d, ", cpu->promenljive[i].ime_promenljive, cpu->promenljive[i].vrednost_promenljive);
16        }
17    }
18    fclose(log_fajl);
19 }

```

Slika 17 – funkcija „upis_Log“

Funkcija „upis_Log“ upisuje stanje centralne procesorske jedinice u log datoteku kreiranu pri pokretanju programa. Svaki upisani status sadrži sekvencijalni broj izvršavanja (korak) kao i unutrašnji kod za instrukciju i same operande.

```

1 void upisiVrednost(CPU *cpu, Operand operand1, int vrednost) {
2     if (operand1.tip_operanda == REGISTAR) {
3         cpu->registri[operand1.registar] = vrednost & MASKA_8bit;
4     } else if (operand1.tip_operanda == PROMENLJIVA) {
5         operand1.promenljiva->vrednost_promenljive = vrednost & MASKA_8bit;
6     } else {
7         prijaviGresku("Nevalidan operand za upisivanje vrednosti.\n");
8     }
9 }

```

Slika 18 – funkcija „upisiVrednost“

Funkcija „upisiVrednost“ upisuje vrednost drugog operandu u prvi operand samo u slučaju da je prvi operand registar ili promenljiva. Svaka upisana vrednost se tumači kao 8-bitni neoznačeni broj.

```

7 void upisiFlagove(CPU *cpu, ImeInstrukcije instrukcija, int inicijalni, int rezultat, int Vrednost_Operand1, int Vrednost_Operand2) {
8     switch(instrukcija) {
9         case ADD:
10             cpu->flagovi[ZF] = (rezultat == 0) ? 1 : 0;
11             cpu->flagovi[SF] = (rezultat & ZNAKOVNI_8bit) != 0 ? 1 : 0;
12             cpu->flagovi[CF] = (inicijalni > MASKA_8bit) ? 1 : 0;
13             cpu->flagovi[OF] = ((Vrednost_Operand1 ^ inicijalni) & (Vrednost_Operand2 ^ inicijalni) & ZNAKOVNI_8bit) != 0 ? 1 : 0;
14             break;
15         case SUB:
16         case CMP:
17             cpu->flagovi[ZF] = (rezultat == 0) ? 1 : 0;
18             cpu->flagovi[SF] = (rezultat & ZNAKOVNI_8bit) != 0 ? 1 : 0;
19             cpu->flagovi[CF] = (Vrednost_Operand1 < Vrednost_Operand2) ? 1 : 0;
20             cpu->flagovi[OF] = ((Vrednost_Operand1 ^ Vrednost_Operand2) & (Vrednost_Operand1 ^ inicijalni) & ZNAKOVNI_8bit) != 0 ? 1 : 0;
21             break;
22         case INC:
23             cpu->flagovi[ZF] = (rezultat == 0) ? 1 : 0;
24             cpu->flagovi[SF] = (rezultat & ZNAKOVNI_8bit) != 0 ? 1 : 0;
25             cpu->flagovi[OF] = (Vrednost_Operand1 == 0x7F) ? 1 : 0;
26             break;
27         case DEC:
28             cpu->flagovi[ZF] = (rezultat == 0) ? 1 : 0;
29             cpu->flagovi[SF] = (rezultat & ZNAKOVNI_8bit) != 0 ? 1 : 0;
30             cpu->flagovi[OF] = (Vrednost_Operand1 == 0x80) ? 1 : 0;
31             break;
32     }
33 }

```

Slika 19 – funkcija „upisiFlagove“

Funkcija „upisiFlagove“ koristi podatke aritmetičkih operacija da odredi stanje flagova u centralnoj procesorskoj jedinici.

Može se primetiti da instrukcije INC i DEC ne postavljaju carry flag. [4]

3. Zaključak

U okviru ovog rada je realizovan pojednostavljen softverski interpreter pseudo-asmblerskog koda edukativne svrhe. Cilj je bio kreiranje aplikacije koja omogućava korisnicima da lakše analiziraju uticaj asmblerskih instrukcija na vrednosti statusnih flagova u procesoru. Realizovani softver demonstrira osnovne koncepte flagova u procesoru i njihovu praktičnu primenu u kontroli toka izvršavanja programa.

4. Literatura

- [1] Andrew S. Tanenbaum, Structured Computer Organization, 6. Izdanje, Pearson, 2013
- [2] William Stallings, Computer Organization and Architecture, Pearson, 2019.
- [3] David A. Patterson, John L. Hennessy, Computer Organization and Design, Morgan Kaufmann, 2021.
- [4] Intel Corporation, Intel 64 and IA-32 Architectures Software Developer's Manual, <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>, datum pristupa: 24.5.2026.
- [5] ARM Limited, ARM Architecture Reference Manual, <https://developer.arm.com/documentation>, datum pristupa: 24.5.2026.
- [6] Dr Željko Stojanov, Praktikum za vežbe iz predmeta Računarski sistemi, Univerzitet u Novom Sadu, 2016. <http://147.91.177.109/Content/files/0/RSpraktikum2016.pdf>, datum pristupa 23.5.2026.
- [7] RISC-V International, The RISC-V Instruction Set Manual, <https://riscv.org/technical/specifications/>, datum pristupa: 25.5.2026.
- [8] Microchip Technology Inc., AVR Microcontrollers Documentation, <https://www.microchip.com/en-us/products/microcontrollers-and-microprocessors/8-bit-mcus/avr-mcus>, datum pristupa: 25.5.2026.
- [9] Microsoft Corporation, WinDbg Documentation, <https://learn.microsoft.com/en-us/windows-hardware/drivers/debugger/>, datum pristupa: 25.5.2026.
- [10] Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces, <https://pages.cs.wisc.edu/~remzi/OSTEP/>, datum pristupa: 25.5.2026.